TRIBHUVAN UNIVERSITY

INSTITUTE OF SCIENCE AND TECHNOLOGY

AMRIT SCIENCE CAMPUS

Data Warehouse and Data Mining (CSC 420)

Lab Report

SUBMITTED BY:

Name: Samchita Pandey
Roll: 79010245
Date: 2083/01/15

SUBMITTED TO:

Dabbal Singh Mahara
Department of CSIT

External Signature

Internal Signature

TABLE OF CONTENT

S.N	Title	Page no	Date	Signature
1	Install Python3(Anaconda) for lab preparation	1		
2	Data preprocessing and cleaning using Python libraries	5		
3	Apriori algorithm and association rules	15		
4	FP-Growth algorithm	17		
5	Classification using the ID3 algorithm	19		
6	Classification using the Bayesian algorithm	21		
7	Classification using SVM algorithm	23		
8	Linear Regression algorithm	25		
9	Multiple Linear Regression algorithm	27		
10	Classification using the Backpropagation algorithm	29		
11	Clustering using the K-Means algorithm	32		
12	Clustering using the K-Medoid algorithm	34		
13	Clustering using Agglomerative (Hierarchical) algorithm	36		
14	Clustering using DBSCAN algorithm	38		

Lab 1: Install Python 3 (Anaconda) for lab preparation.

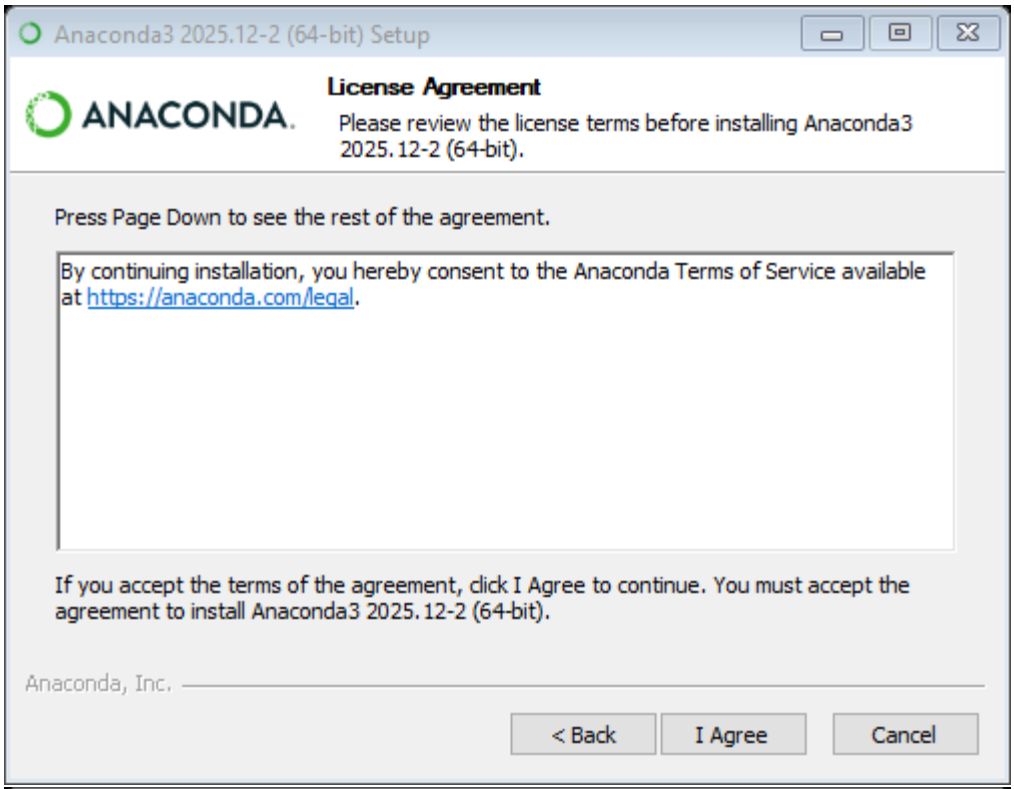
Introduction

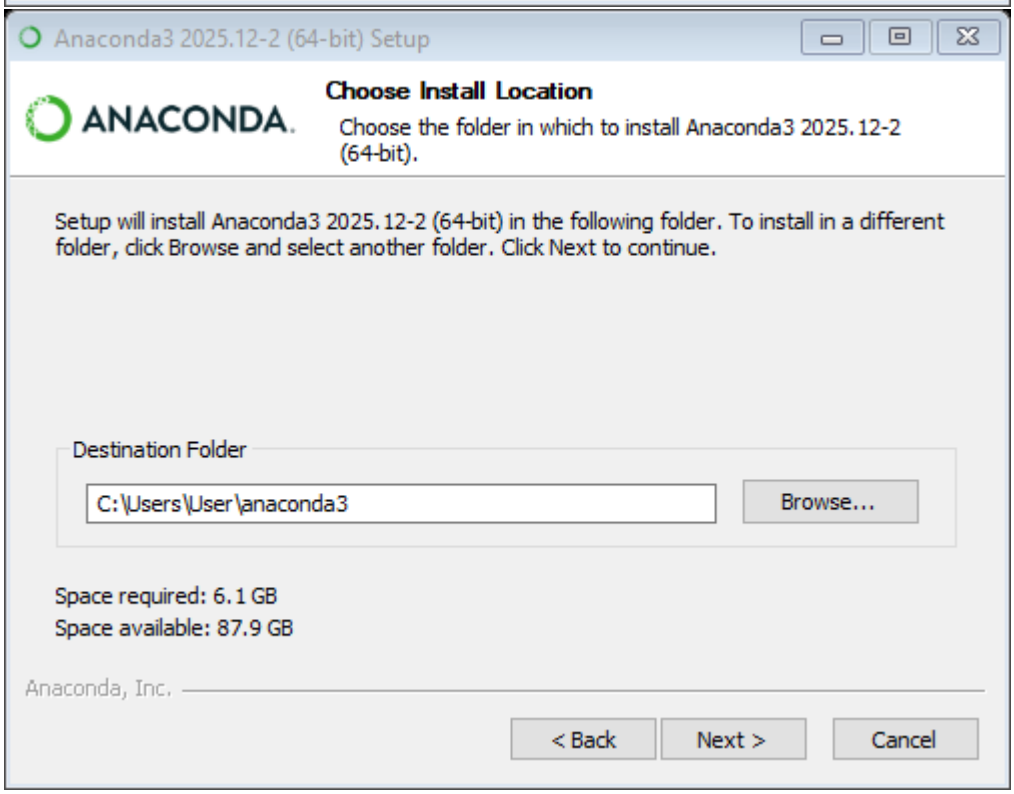
Python is widely used in data mining because of its rich ecosystem of libraries for preprocessing, machine learning, and visualization. Anaconda is a distribution that simplifies Python package and environment management through conda.

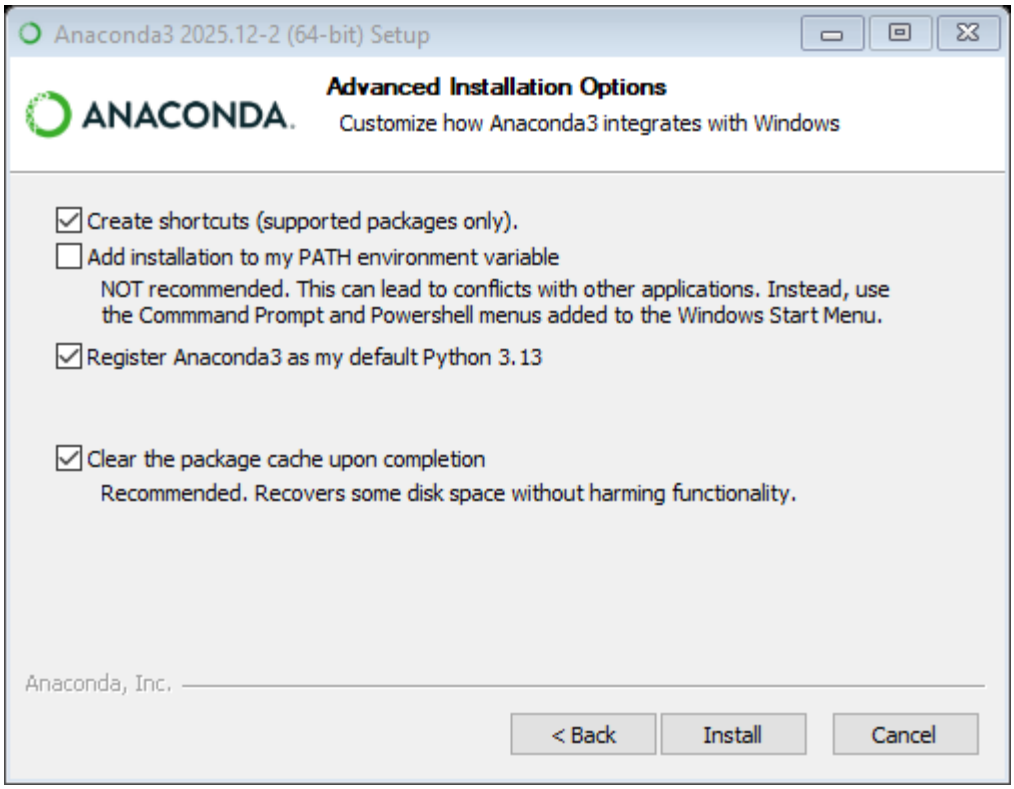
Steps for installation

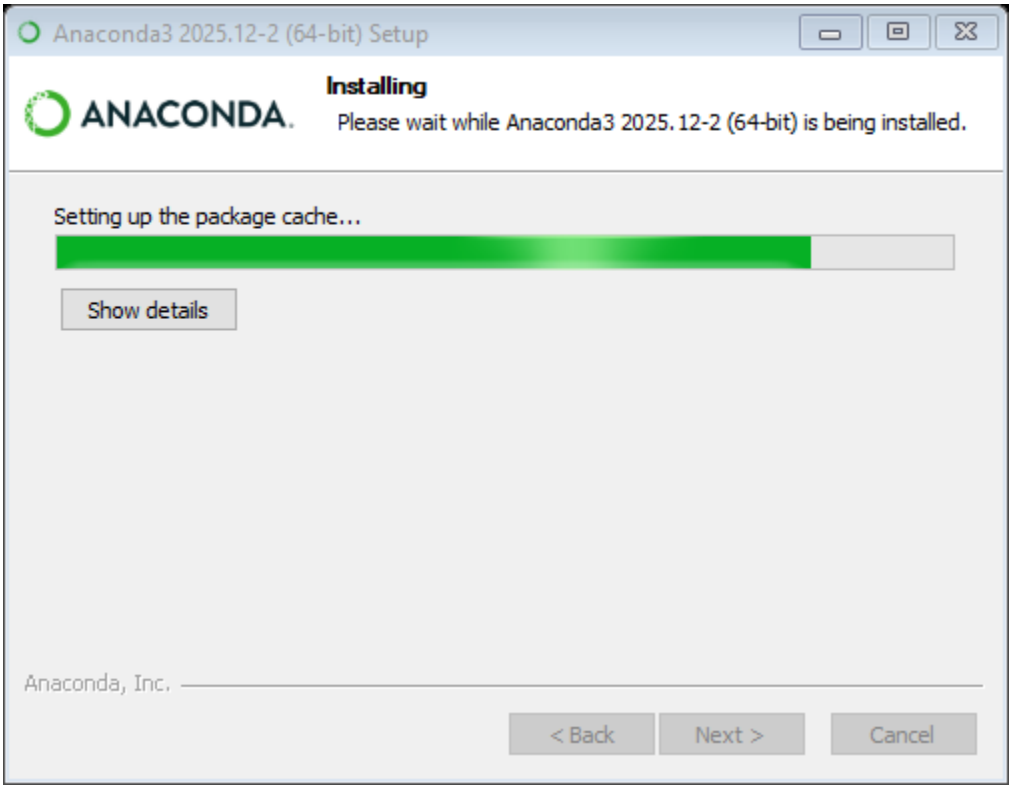
1. Download Anaconda from the official website: - https://www.anaconda.com/download
2. Install Anaconda on your system. Run the downloaded installer from the terminal.

Follow the on-screen instructions as shown in the images below:


Anaconda3 2025.12-2 (64-bit) Setup
ANACONDA.
License Agreement
Please review the license terms before installing Anaconda3 2025.12-2 (64-bit).
Press Page Down to see the rest of the agreement.
By continuing installation, you hereby consent to the Anaconda Terms of Service available at https://anaconda.com/legal.
If you accept the terms of the agreement, click I Agree to continue. You must accept the agreement to install Anaconda3 2025.12-2 (64-bit).
Anaconda, Inc.
< Back
I Agree
Cancel


Anaconda3 2025.12-2 (64-bit) Setup
ANACONDA.
Choose Install Location
Choose the folder in which to install Anaconda3 2025.12-2 (64-bit).
Setup will install Anaconda3 2025.12-2 (64-bit) in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.
Destination Folder
C:\Users\User\anaconda3
Browse...
Space required: 6.1 GB
Space available: 87.9 GB
Anaconda, Inc.
< Back
Next >
Cancel


Anaconda3 2025.12-2 (64-bit) Setup
ANACONDA.
Advanced Installation Options
Customize how Anaconda3 integrates with Windows
Create shortcuts (supported packages only).
Add installation to my PATH environment variable
NOT recommended. This can lead to conflicts with other applications. Instead, use the Commmand Prompt and Powershell menus added to the Windows Start Menu.
Register Anaconda3 as my default Python 3.13
Clear the package cache upon completion
Recommended. Recovers some disk space without harming functionality.
Anaconda, Inc.
< Back
Install
Cancel


Anaconda3 2025.12-2 (64-bit) Setup
ANACONDA.
Installing
Please wait while Anaconda3 2025.12-2 (64-bit) is being installed.
Setting up the package cache...
Show details
Anaconda, Inc.
< Back
Next >
Cancel

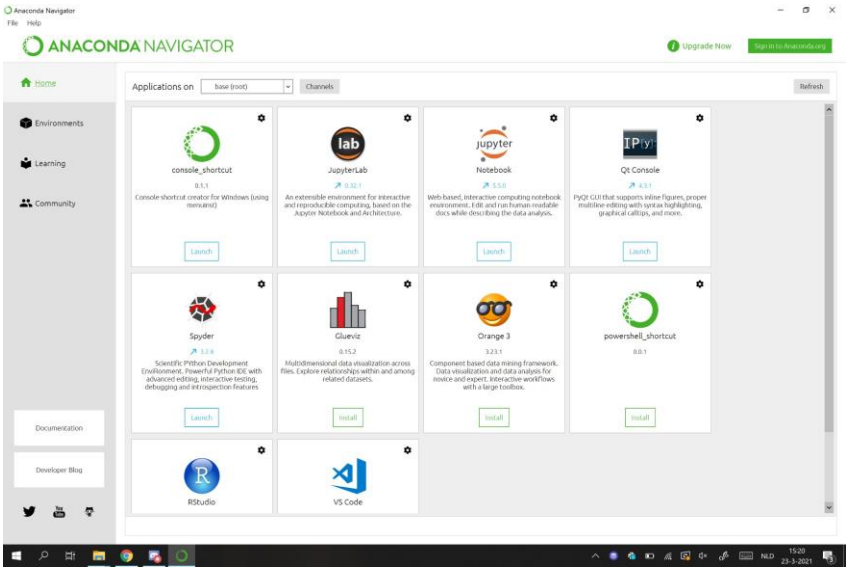


Figure 1: Anaconda Installation Setup Window

3. Open Anaconda Prompt or terminal and verify installation:

python–version conda–version

4. Then create a new environment for lab work:
conda create-n dwdm python=3.11 conda activate dwdm

5. Install required packages: pip install numpy pandas scikit-learn mlxtend

Lab 2: Demonstrate the concept of data preprocessing and cleaning using Python libraries.

Introduction

Data preprocessing is the process of transforming raw data into a clean and usable format before applying any machine learning algorithm. An actual real-world data is often incomplete with missing values, noisy with errors and outliers and inconsistent with various different formats. Therefore, preprocessing is essential to improve accuracy and performance of models.

Data cleaning is a part of preprocessing that focuses on detecting and correcting errors in the dataset. The common cleaning tasks involves handling missing values, removing duplicate data , fixing inconsistent formats and detecting and removing outliers.

Algorithm

1. First, we create the raw dataset in a DataFrame.
2. We clean the noisy text fields by standardizing the string values.
3. We remove duplicate records and replace missing values by using the median or mode.4. We remove any remaining missing records and filter outliers from the numeric columns.
5. We discretize the age and income columns and encode the purchased labels.
6. Finally, we scale the income column and display the processed data with its summary.

Implementation

```
import pandas as pd
pd.set_option("display.max_columns", None)   # show all columns
pd.set_option("display.width", 120)          # wider terminal output
# ── Step 1: Noise handling
──────────────────────────────────────────────────────────────────────────────
def noise_handle_data(df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.copy()
    cleaned["city"] = cleaned["city"].astype("string").str.strip().str.title()
    cleaned["purchased"] = (
        cleaned["purchased"].astype("string").str.strip().str.lower()
    )
    return cleaned
# ── Step 2: Remove duplicates
──────────────────────────────────────────────────────────────────────────────
def remove_duplication(df: pd.DataFrame) -> pd.DataFrame:
    return df.drop_duplicates()
```

```
# ── Step 3: Fill missing values
──────────────────────────────────────────────────────────────────────────────
def replace_missing_value_by_median(df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.copy()
    for col in ["age", "income"]:
        cleaned[col] = cleaned[col].fillna(cleaned[col].median())
    cleaned["city"] = cleaned["city"].fillna(cleaned["city"].mode()[0])
    return cleaned
# ── Step 4: Drop remaining NaN
──────────────────────────────────────────────────────────────────────────────
def remove_missing_data(df: pd.DataFrame) -> pd.DataFrame:
    return df.dropna()
# ── Step 5: Outlier removal (IQR)
──────────────────────────────────────────────────────────────────────────────
def outlier(df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.copy()
    for col in ["age", "income"]:
        q1 = cleaned[col].quantile(0.25)
        q3 = cleaned[col].quantile(0.75)
        iqr = q3 - q1
        lower = q1 - 1.5 * iqr
        upper = q3 + 1.5 * iqr
        cleaned = cleaned[(cleaned[col] >= lower) & (cleaned[col] <= upper)]
    return cleaned

# ── Step 6: Discretization
──────────────────────────────────────────────────────────────────────────────
def discretization(df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.copy()
    cleaned["age_group"] = pd.cut(
       cleaned["age"],
        bins=[0, 21, 24, 100],
        labels=["Young", "Adult", "Senior"]
    )
    cleaned["income_group"] = pd.cut(
        cleaned["income"],
        bins=[0, 27000, 31000, 100000],
        labels=["Low", "Medium", "High"],
    )
    return cleaned
# ── Display helper
──────────────────────────────────────────────────────────────────────────────
```

```
────────
def section(title: str) -> None:
    print("\n" + "=" * 60)
    print(f"  {title}")
    print("=" * 60)

def view(df: pd.DataFrame) -> None:
    section("FINAL PROCESSED DATA — All Rows & Columns")
    print(df.to_string(index=True))

    section("COLUMN-BY-COLUMN VALUES")
    for col in df.columns:
        print(f"\n  [{col}]")
        for i, val in enumerate(df[col].values):
            print(f"    Row {i}: {val}")
    section("STATISTICAL SUMMARY")
    print(df.describe(include="all").to_string())
    section("DATA TYPES & NON-NULL COUNTS")
    df.info()

# ── Main pipeline
────────────────────────────────────────────────────────────────────────────────
──────────
def main() -> None:
    # Raw dataset (embedded — no file needed)
    data = pd.DataFrame({
        "name":      ["Asha", "Bikash", "Asha", "Chandra", "Dipesh", "Elina"],
        "age":       [21, None, 21, 24, 23, None],
        "income":    [25000, 32000, 25000, None, 28000, 31000],
        "city":      ["Kathmandu", "pokhara", "Kathmandu", "Lalitpur ", None, "pokhara"],
        "purchased": ["Yes", "No", "Yes", "No", "Yes", "No"],
    })

    section("RAW DATA (before any processing)")
    print(data.to_string(index=True))
    print(f"\nMissing values:\n{data.isnull().sum()}")

    # Pipeline
    processed = noise_handle_data(data)
    section("AFTER STEP 1 — Noise Handling (text cleaned)")
    print(processed[["name", "city", "purchased"]].to_string())
    processed = remove_duplication(processed)
```

```
    section("AFTER STEP 2 — Duplicates Removed")
    print(f"  Rows remaining: {len(processed)}")
    print(processed.to_string())

    processed = replace_missing_value_by_median(processed)

    section("AFTER STEP 3 — Missing Values Filled")
    print(processed.to_string())

   processed = remove_missing_data(processed)
    section("AFTER STEP 4 — Remaining NaN Dropped")
    print(processed.to_string())

    processed = outlier(processed)
    section("AFTER STEP 5 — Outliers Removed")
    print(processed.to_string())

    processed = discretization(processed)
    section("AFTER STEP 6 — Discretization")
    print(processed[["name", "age", "age_group", "income", "income_group"]].to_string())
    # Encoding & scaling
   processed["purchased"] = processed["purchased"].map({"yes": 1, "no": 0})
    processed["income_scaled"] = (
        (processed["income"] - processed["income"].min())
        / (processed["income"].max() - processed["income"].min())
    )

   # Final rich view
    view(processed)
if __name__ == "__main__":
    main()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python Lab1data_preprocessing.py

============================================================
  RAW DATA (before any processing)
============================================================
      name   age   income       city purchased
0     Asha  21.0  25000.0  Kathmandu       Yes
1   Bikash   NaN  32000.0    pokhara        No
2     Asha  21.0  25000.0  Kathmandu       Yes
3  Chandra  24.0      NaN  Lalitpur        No
4   Dipesh  23.0  28000.0        NaN       Yes
5    Elina   NaN  31000.0    pokhara        No

Missing values:
name         0
age          2
income       1
city         1
purchased    0
dtype: int64

============================================================
  AFTER STEP 1 — Noise Handling (text cleaned)
============================================================
      name       city purchased
0     Asha  Kathmandu       yes
1   Bikash    Pokhara        no
2     Asha  Kathmandu       yes
3  Chandra   Lalitpur        no
4   Dipesh       <NA>       yes
5    Elina    Pokhara        no
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python Lab1data_preprocessing.py
  AFTER STEP 2 — Duplicates Removed
============================================================
  Rows remaining: 5
      name   age   income       city purchased
0     Asha  21.0  25000.0  Kathmandu       yes
1   Bikash   NaN  32000.0    Pokhara        no
3  Chandra  24.0      NaN   Lalitpur        no
4   Dipesh  23.0  28000.0       <NA>       yes
5    Elina   NaN  31000.0    Pokhara        no

============================================================
  AFTER STEP 3 — Missing Values Filled
============================================================
      name   age   income       city purchased
0     Asha  21.0  25000.0  Kathmandu       yes
1   Bikash  23.0  32000.0    Pokhara        no
3  Chandra  24.0  29500.0   Lalitpur        no
4   Dipesh  23.0  28000.0    Pokhara       yes
5    Elina  23.0  31000.0    Pokhara        no

============================================================
  AFTER STEP 4 — Remaining NaN Dropped
============================================================
      name   age   income       city purchased
0     Asha  21.0  25000.0  Kathmandu       yes
1   Bikash  23.0  32000.0    Pokhara        no
3  Chandra  24.0  29500.0   Lalitpur        no
4   Dipesh  23.0  28000.0    Pokhara       yes
5    Elina  23.0  31000.0    Pokhara        no
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python Lab1data_preprocessing.py

============================================================
  AFTER STEP 5 — Outliers Removed
============================================================
     name   age   income     city purchased
1  Bikash  23.0  32000.0  Pokhara        no
4  Dipesh  23.0  28000.0  Pokhara       yes
5   Elina  23.0  31000.0  Pokhara        no

============================================================
  AFTER STEP 6 — Discretization
============================================================
     name   age age_group   income income_group
1  Bikash  23.0     Adult  32000.0         High
4  Dipesh  23.0     Adult  28000.0       Medium
5   Elina  23.0     Adult  31000.0       Medium

============================================================
  FINAL PROCESSED DATA — All Rows & Columns
============================================================
     name   age   income     city  purchased age_group income_group  income_scaled
1  Bikash  23.0  32000.0  Pokhara          0     Adult         High           1.00
4  Dipesh  23.0  28000.0  Pokhara          1     Adult       Medium           0.00
5   Elina  23.0  31000.0  Pokhara          0     Adult       Medium           0.75

============================================================
  COLUMN-BY-COLUMN VALUES
============================================================

  [name]
    Row 0: Bikash
    Row 1: Dipesh
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python Lab1data_preprocessing.py

  [name]
    Row 0: Bikash
    Row 1: Dipesh
    Row 2: Elina

  [age]
    Row 0: 23.0
    Row 1: 23.0
    Row 2: 23.0

  [income]
    Row 0: 32000.0
    Row 1: 28000.0
    Row 2: 31000.0

  [city]
    Row 0: Pokhara
    Row 1: Pokhara
    Row 2: Pokhara

  [purchased]
    Row 0: 0
    Row 1: 1
    Row 2: 0

  [age_group]
    Row 0: Adult
    Row 1: Adult
    Row 2: Adult

  [income_group]
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python Lab1data_preprocessing.py
  [income_group]
    Row 0: High
    Row 1: Medium
    Row 2: Medium

  [income_scaled]
    Row 0: 1.0
    Row 1: 0.0
    Row 2: 0.75

============================================================
  STATISTICAL SUMMARY
============================================================
          name   age        income     city  purchased age_group income_group  income_sc
aled
count        3   3.0      3.000000        3   3.000000         3            3       3.00
0000
unique       3   NaN           NaN        1        NaN         1            2
 NaN
top     Bikash   NaN           NaN  Pokhara        NaN     Adult       Medium
 NaN
freq         1   NaN           NaN        3        NaN         3            2
 NaN
mean       NaN  23.0  30333.333333      NaN   0.333333       NaN          NaN       0.58
3333
std        NaN   0.0   2081.665999      NaN   0.577350       NaN          NaN       0.52
0416
min        NaN  23.0  28000.000000      NaN   0.000000       NaN          NaN       0.00
0000
25%        NaN  23.0  29500.000000      NaN   0.000000       NaN          NaN       0.37
5000
50%        NaN  23.0  31000.000000      NaN   0.000000       NaN          NaN       0.75
0000
75%        NaN  23.0  31500.000000      NaN   0.500000       NaN          NaN       0.87
max        NaN  23.0  32000.000000      NaN   1.000000       NaN          NaN       1.00
0000
```

```
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python Lab1data_preprocessing.py
0000

============================================================
  DATA TYPES & NON-NULL COUNTS
============================================================
<class 'pandas.DataFrame'>
Index: 3 entries, 1 to 5
Data columns (total 8 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   name           3 non-null      str
 1   age            3 non-null      float64
 2   income         3 non-null      float64
 3   city           3 non-null      string
 4   purchased      3 non-null      int64
 5   age_group      3 non-null      category
 6   income_group   3 non-null      category
 7   income_scaled  3 non-null      float64
dtypes: category(2), float64(3), int64(1), str(1), string(1)
memory usage: 438.0 bytes
```

Lab 3: Write a program to implement Apriori algorithm and generate association rules.

Introduction

The Apriori algorithm is a widely used data mining technique for discovering frequent itemsets and generating association rules from a dataset. It is commonly applied in market basket analysis, where it helps identify relationships between items purchased together. The algorithm works on the fundamental principle that: "If an itemset is frequent, then all of its subsets must also be frequent."

Algorithm

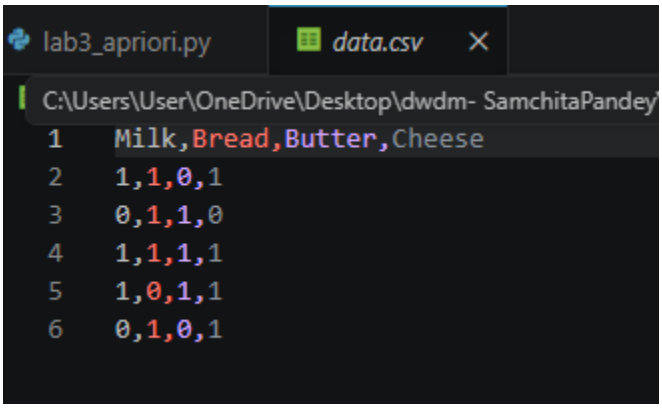
1. First, we encode the transactions into one-hot format.

2. We find the frequent itemsets with the minimum support.

3. We generate the association rules from the frequent itemsets.

4. We filter the rules by using the confidence threshold

5. Finally, we print the support, confidence, and lift values.



Implementation

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
# Load CSV
df = pd.read_csv("data.csv")
# Apply Apriori
frequent_items = apriori(df, min_support=0.4, use_colnames=True)
```

```
# Generate rules
rules = association_rules(frequent_items, metric="confidence",
min_threshold=0.6)

print(frequent_items)
print(rules)
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   powershell

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab3_apriori.py
future.Please use a DataFrame with bool type
  warnings.warn(
    support                          itemsets
0       0.6                 frozenset({Milk})
1       0.8                frozenset({Bread})
2       0.6               frozenset({Butter})
3       0.8               frozenset({Cheese})
4       0.4          frozenset({Bread, Milk})
5       0.4         frozenset({Butter, Milk})
6       0.6         frozenset({Cheese, Milk})
7       0.4        frozenset({Butter, Bread})
8       0.6        frozenset({Cheese, Bread})
9       0.4       frozenset({Cheese, Butter})
10      0.4   frozenset({Cheese, Bread, Milk})
11      0.4  frozenset({Cheese, Butter, Milk})
                   antecedents                 consequents  antecedent support  ...   jaccard  certainty  kulczynski
0            frozenset({Milk})           frozenset({Bread})                 0.6  ...  0.400000  -0.666667    0.583333
1          frozenset({Butter})            frozenset({Milk})                 0.6  ...  0.500000   0.166667    0.666667
2            frozenset({Milk})          frozenset({Butter})                 0.6  ...  0.500000   0.166667    0.666667
3          frozenset({Cheese})            frozenset({Milk})                 0.8  ...  0.750000   0.375000    0.875000
4            frozenset({Milk})          frozenset({Cheese})                 0.6  ...  0.750000   1.000000    0.875000
5          frozenset({Butter})           frozenset({Bread})                 0.6  ...  0.400000  -0.666667    0.583333
6          frozenset({Cheese})           frozenset({Bread})                 0.8  ...  0.600000  -0.250000    0.750000
7           frozenset({Bread})          frozenset({Cheese})                 0.8  ...  0.600000  -0.250000    0.750000
8          frozenset({Butter})          frozenset({Cheese})                 0.6  ...  0.400000  -0.666667    0.583333
9   frozenset({Cheese, Bread})            frozenset({Milk})                 0.6  ...  0.500000   0.166667    0.666667
10   frozenset({Cheese, Milk})           frozenset({Bread})                 0.6  ...  0.400000  -0.666667    0.583333
11    frozenset({Bread, Milk})          frozenset({Cheese})                 0.4  ...  0.500000   1.000000    0.750000
12           frozenset({Milk})   frozenset({Cheese, Bread})                 0.6  ...  0.500000   0.166667    0.666667
13  frozenset({Cheese, Butter})            frozenset({Milk})                 0.4  ...  0.666667   1.000000    0.833333
14   frozenset({Cheese, Milk})          frozenset({Butter})                 0.6  ...  0.500000   0.166667    0.666667
15   frozenset({Butter, Milk})          frozenset({Cheese})                 0.4  ...  0.500000   1.000000    0.750000
16         frozenset({Butter})    frozenset({Cheese, Milk})                 0.6  ...  0.500000   0.166667    0.666667
17           frozenset({Milk})  frozenset({Cheese, Butter})                 0.6  ...  0.666667   0.444444    0.833333

[18 rows x 14 columns]
```

Lab 4: Write a program to implement the FP-Growth algorithm

Theory

The FP-Growth (Frequent Pattern Growth) algorithm is an efficient data mining technique used to find frequent itemsets without generating candidate sets, unlike the Apriori algorithm. It uses a special data structure called an FP-Tree (Frequent Pattern Tree) to store the dataset in a compressed form. It is mainly based on idea that instead of generating candidates repeatedly (like Apriori), FP-Growth builds a tree and extracts patterns directly. FP-Growth Algorithm are used for Market basket analysis, recommendation systems and pattern mining.

Algorithm

1. Scan Dataset by reading transaction data and calculate support count of each item
2. Remove Infrequent Items whose support < minimum support
3. Sort items in descending order of support
4. Build FP-Tree by creating root node (NULL), inserting transactions into tree and maintain header table for item links
5. Construct Conditional Pattern Base for each item by extracting paths leading to that item
6. Build Conditional FP-Tree and construct smaller FP-Trees for each item
7. Generate Frequent Itemsets by recursively mining the FP-Trees and generate all frequent patterns
8. Display frequent itemsets

Implementation

```
import pandas as pd
from mlxtend.frequent_patterns import fpgrowth
# Step 1: Create dataset
data = {
    'Milk':   [1, 0, 1, 1, 0],
    'Bread':  [1, 1, 1, 0, 1],
    'Butter': [0, 1, 1, 1, 0],
    'Cheese': [1, 0, 1, 1, 1]
}
df = pd.DataFrame(data)
# Step 2: Apply FP-Growth
frequent_items = fpgrowth(df, min_support=0.4, use_colnames=True)
# Step 3: Display result
print("Frequent Itemsets:\n", frequent_items)
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab4_FP-Growth.py
Frequent Itemsets:
     support                          itemsets
0        0.8                frozenset({Cheese})
1        0.8                 frozenset({Bread})
2        0.6                  frozenset({Milk})
3        0.6                frozenset({Butter})
4        0.6          frozenset({Bread, Cheese})
5        0.6           frozenset({Cheese, Milk})
6        0.4            frozenset({Bread, Milk})
7        0.4           frozenset({Butter, Milk})
8        0.4    frozenset({Bread, Cheese, Milk})
9        0.4   frozenset({Butter, Cheese, Milk})
10       0.4          frozenset({Bread, Butter})
11       0.4         frozenset({Butter, Cheese})
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey>
```

Lab 5: Write a program to demonstrate classification using the ID3 algorithm

Introduction

The ID3 (Iterative Dichotomiser 3**)** algorithm is a popular decision tree classification algorithm used in machine learning. It builds a tree by selecting the best attribute that separates the data into classes. ID3 selects the attribute with the highest information gain to create decision nodes. The main concept used in ID3 is Entropy and Information Gain where Entropy measures impurity or randomness in data and Information Gain measures how well an attribute splits the data .ID3 algorithm can be used for classification problems, decision-making systems, medical diagnosis and spam detection.

Algorithm

1. Calculate entropy of dataset
2. Compute information gain for each feature
3. Select best feature (highest gain)
4. Split data
5. Repeat until stopping condition
6. Predict class (Spam / Not Spam)

Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Step 1: Sample dataset (1 = word present, 0 = not present)
# Features: [Free, Win, Offer, Click]
X = [
    [1, 1, 1, 1],  # Spam
    [1, 0, 1, 1],  # Spam
    [0, 1, 0, 1],  # Spam
    [0, 0, 0, 0],  # Not Spam
    [0, 0, 1, 0],  # Not Spam
    [0, 1, 0, 0]   # Not Spam
]

# Labels: 1 = Spam, 0 = Not Spam
y = [1, 1, 1, 0, 0, 0]
# Step 2: Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Step 3: Train ID3 model (using entropy)
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X_train, y_train)
# Step 4: Predict
y_pred = model.predict(X_test)
# Step 5: Accuracy
print("Predictions:", y_pred)
print("Actual:", y_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
# Step 6: Test with new message
# Example: "Free Offer Click"
new_msg = [[1, 0, 1, 1]]
result = model.predict(new_msg)

if result[0] == 1:
    print("Message is SPAM")
else:
    print("Message is NOT SPAM")
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab5-ID3.py
Predictions: [1 1]
Actual: [1, 1]
Accuracy: 1.0
Message is SPAM
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey>
```

Lab 6: Write a program to demonstrate classification using the Bayesian algorithm.

Introduction

The Bayesian algorithm, commonly known as Naive Bayes, is a probabilistic classification method based on Bayes' Theorem. It predicts the class of data based on the probability of features. It is called *naive* because it assumes that all features are independent of each other. It calculates the probability of a class given the input features and selects the class with the highest probability. It can be used for spam detection, text classification, medical diagnosis and sentiment analysis.

Algorithm

Step 1: Collect Dataset

- Gather patient data with features (age, BP, glucose, etc.)
- Each record has a **class label**:
  - Disease = 1
  - No Disease = 0

Step 2: Preprocess Data like handle missing values, convert data into numeric form and split dataset into training and testing sets

Step 3: Calculate Prior Probabilities like P(Disease) and P(No Disease)

Step 4: Calculate Likelihood for each feature like P(Feature|Disease)

Step 5: Apply Bayes' Theorem and compute:

P(Disease|Features)

Step 6: Predict disease and no disease by comparing probabilities:

- If P(Disease|X) is higher → Disease
- Else → No Disease

Step 7: Evaluate Model by calculating accuracy using test data

Step 8: Output

- Display prediction and accuracy

Implementation

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
# Step 1: Create sample medical dataset
data = {
    'Age': [25, 45, 35, 50, 23, 40, 60, 48],
    'BP': [120, 140, 130, 150, 110, 135, 160, 145],
    'Glucose': [85, 130, 120, 150, 90, 140, 160, 155],
    'Cholesterol': [180, 220, 200, 240, 170, 210, 260, 230],
    'Disease': [0, 1, 0, 1, 0, 1, 1, 1]
}
df = pd.DataFrame(data)
# Step 2: Split features and target
X = df[['Age', 'BP', 'Glucose', 'Cholesterol']]
y = df['Disease']
# Step 3: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
# Step 4: Train model
model = GaussianNB()
model.fit(X_train, y_train)
# Step 5: Prediction
y_pred = model.predict(X_test)
# Step 6: Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
# Step 7: Predict new patient
new_patient = [[45, 140, 135, 220]]
result = model.predict(new_patient)
if result[0] == 1:
    print("Patient is likely to have disease")
else:
    print("Patient is likely healthy")
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python  lab6-BayesianAlgorithm.py
Accuracy: 1.0
Patient is likely to have disease
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey>
```

Lab 7: Write a program to demonstrate classification using SVM algorithm

Introduction

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression problems, but it is mostly used for classification. SVM finds the best line that separates classes with maximum gap. SVM can be used for spam email detection, image classification, face recognition and disease diagnosis.

Algorithm

1. Import dataset
2. Split dataset into training and testing data
3. Choose SVM classifier
4. Train the model using training data
5. Predict output using test data
6. Measure accuracy
7. Display results

Implementation

```
from sklearn import metrics
from sklearn.svm import SVC

def main() -> None:
    food = {
        "data": [
            [220, 12, 8, 5],
            [180, 9, 5, 4],
            [320, 14, 18, 6],
            [150, 7, 4, 3],
            [410, 20, 22, 8],
            [95, 4, 1, 2],
            [130, 6, 3, 2],
            [360, 16, 19, 7],
            [280, 11, 10, 5],
            [70, 3, 1, 1],
           [240, 10, 9, 4],
            [390, 18, 21, 7],
        ],
        "target": [1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0],
```

```
        "feature_names": ["Calories", "Protein", "Fat", "Sugar"],
        "target_names": ["Unhealthy", "Healthy"],
    }
    x = food["data"]
    y = food["target"]
    print("Length of Data:", len(x))
    split = int(len(x) * 0.7)
    trainx, testx = x[:split], x[split:]
    trainy, testy = y[:split], y[split:]
    print("Number of features:", len(food["feature_names"]))
    print("Number of classes: ", len(food["target_names"]))
    print("Class Labels:      ", food["target_names"])
    model = SVC(kernel="linear")
    model.fit(trainx, trainy)
    yp = model.predict(testx)
    print("\nConfusion Matrix:")
    print(metrics.confusion_matrix(testy, yp))
    print("\nClassification Measures:")
    print("Accuracy: ", metrics.accuracy_score(testy, yp))
    print("Recall:   ", metrics.recall_score(testy, yp, zero_division=0))   # FIX 3: added
zero_division=0
   print("Precision:", metrics.precision_score(testy, yp, zero_division=0)) # FIX 3: added
zero_division=0
    print("F1-score: ", metrics.f1_score(testy, yp, zero_division=0))       # FIX 3: added
zero_division=0
if __name__ == "__main__":   # FIX 4: added indentation to the guard block
    main()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab7-SVM.py
Classification Measures:
Accuracy:  0.75
Recall:    0.6666666666666666
Precision: 1.0
F1-score:  0.8
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey>
```

Lab 8: Write a program to implement Linear Regression algorithm.

Introduction

Linear Regression is a supervised machine learning algorithm used to predict a continuous value (number). It works by finding a straight line that best fits the data.

Algorithm

1. Import required libraries
2. Load dataset
3. Separate features (X) and target (y)
4. Split data into training and testing sets
5. Create Linear Regression model
6. Train the model using training data
7. Predict output using test data
8. Evaluate model performance

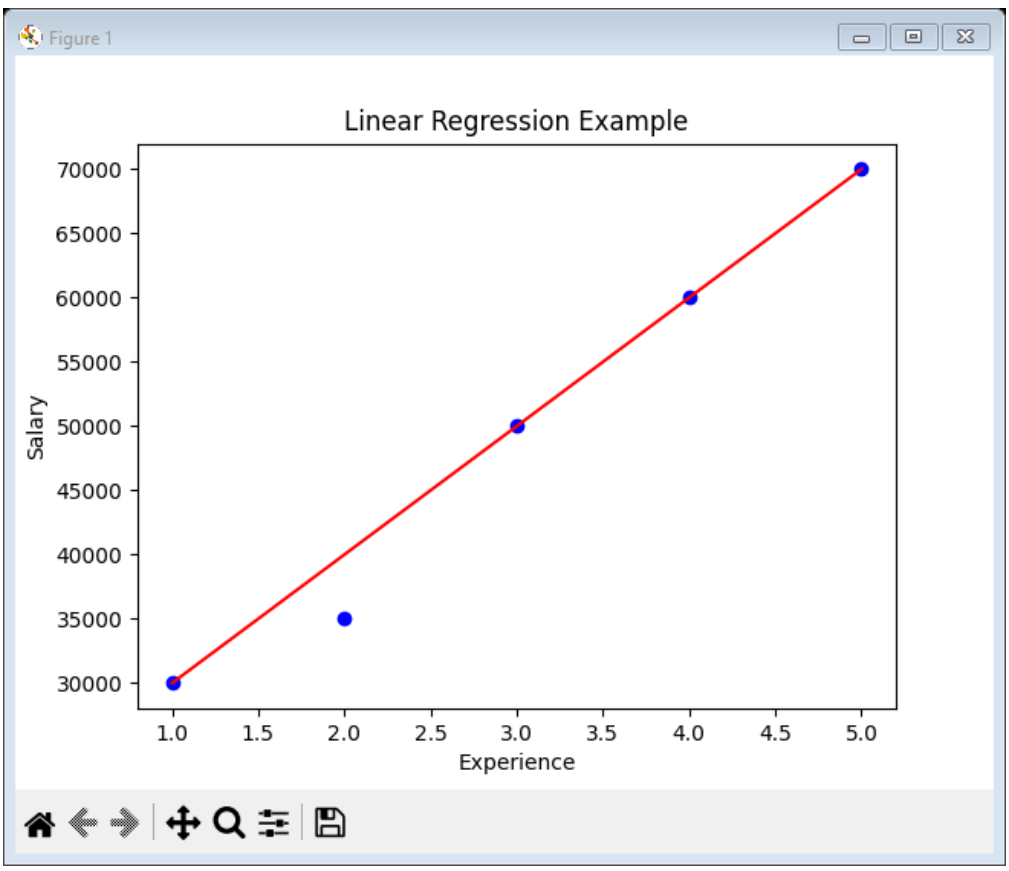
Implementation

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Sample dataset (Experience vs Salary)
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)  # feature
y = np.array([30000, 35000, 50000, 60000, 70000])  # target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create model
model = LinearRegression()
# Train model
model.fit(X_train, y_train)
 y_pred = model.predict(X_test)
# Output results
```

```
print("Actual value:", y_test)
print("Predicted value:", y_pred)
plt.scatter(X, y, color='blue')        # actual data
plt.plot(X, model.predict(X), color='red')  # regression line
plt.title("Linear Regression Example")
plt.xlabel("Experience")
plt.ylabel("Salary")
plt.show()
```

Output

```
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab8-LinearRegression.py
Matplotlib is building the font cache; this may take a moment.
Actual value: [35000]
Predicted value: [40000.]
```



Lab 9: Write a program to implement Multiple Linear Regression algorithm

Introduction

Multiple Linear Regression is an extension of Linear Regression where we use more than one independent variable (feature) to predict a dependent variable. For example: it predict house price using size, number of rooms and location.

Algorithm

1. Import required libraries
2. Load or create dataset
3. Separate features (X) and target (y)
4. Split dataset into training and testing sets
5. Create Multiple Linear Regression model
6. Train the model using training data
7. Predict output using test data
8. Evaluate results

Implementation

```
# Import libraries
import numpy as np
from sklearn.model_selection import train_test_split
rom sklearn.linear_model import LinearRegression

# Sample dataset
# Features: [Size (sq ft), Number of Bedrooms]
X = np.array([
    [1000, 2],
    [1500, 3],
    [2000, 4],
    [2500, 4],
    [3000, 5]
])
# Target: House Price
y = np.array([200000, 300000, 400000, 450000, 500000])

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Create model
model = LinearRegression()
# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Output
print("Actual values:", y_test)
print("Predicted values:", y_pred)
# Show coefficients
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab9-MultipleRegression.py
Actual values: [300000]
Predicted values: [300000.]
Coefficients: [   80. 50000.]
Intercept: 30000.000000000116
```

Lab 10: Write a program to demonstrate classification using the Backpropagation algorithm.

Introduction

Backpropagation is a supervised learning algorithm used to train Artificial Neural Networks (ANNs) for tasks like classification. It works by forward passing input through the network, calculating error (difference between actual and predicted output) and sending the error backward to update weights. This process repeats until the model learns correctly. Backpropagation algorithm has application in the field of image recognition, speech recognition, medical diagnosis and pattern classification.

Algorithm

1. Initialize network weights randomly
2. Input training data into the network
3. Perform forward propagation to get output
4. Compute error (Actual – Predicted)
5. Perform backward propagation (update weights)
6. Repeat steps 2–5 until error is minimized
7. Test the trained model

Implementation

```
Import numpy as np
class NeuralNetwork

    def __init__(self, input_size: int, hidden_size: int, output_size: int,
                 learning_rate: float = 0.5):
        rng = np.random.default_rng(42)
        self.learning_rate = learning_rate
        self.w1 = rng.normal(scale=1.0, size=(input_size, hidden_size))
       self.b1 = np.zeros((1, hidden_size))
        self.w2 = rng.normal(scale=1.0, size=(hidden_size, output_size))
        self.b2 = np.zeros((1, output_size))

    @staticmethod
    def sigmoid(x):
        return 1 / (1 + np.exp(-x))


    @staticmethod
    def sigmoid_derivative(output):
```

```
        return output * (1 - output)

    def forward(self, x):
        self.z1 = np.dot(x, self.w1) + self.b1
        self.a1 = self.sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.w2) + self.b2
        self.a2 = self.sigmoid(self.z2)
        return self.a2

    def train(self, x, y, epochs: int = 10000):
        for epoch in range(epochs):
            output = self.forward(x)
            error = y – output
            loss = np.mean(np.square(error))
            d_output = error * self.sigmoid_derivative(output)
            d_hidden = np.dot(d_output, self.w2.T) * self.sigmoid_derivative(self.a1)
            self.w2 += np.dot(self.a1.T, d_output) * self.learning_rate
            self.b2 += np.sum(d_output, axis=0, keepdims=True) * self.learning_rate  # FIX 2:
broken `.learning_rate` across lines — joined
            self.w1 += np.dot(x.T, d_hidden) * self.learning_rate
            self.b1 += np.sum(d_hidden, axis=0, keepdims=True) * self.learning_rate  # FIX 2: same
fix applied here

            if epoch % 2000 == 0:
                print(f"Epoch {epoch:5d} | Loss: {loss:.6f}")
    def predict(self, x):
        return self.forward(x)

def main() -> None:
    x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
    y = np.array([[0], [1], [1], [0]], dtype=float)

    model = NeuralNetwork(input_size=2, hidden_size=2, output_size=1,
                          learning_rate=0.5)
    model.train(x, y, epochs=10000)


    predictions = model.predict(x)
    print()
    print("Final predictions:")
    for sample, prediction in zip(x, predictions):
        print(f"Input: {sample.astype(int).tolist()} -> Output: {prediction[0]:.4f}")
if __name__ == "__main__":
```

main()

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab10-Backtracking.py
Epoch  4000 | Loss: 0.001681
Epoch  6000 | Loss: 0.000989
Epoch  8000 | Loss: 0.000696

Final predictions:
Input: [0, 0] -> Output: 0.0222
Input: [0, 1] -> Output: 0.9750
Input: [1, 0] -> Output: 0.9752
Input: [1, 1] -> Output: 0.0201
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey>
```

Lab 11: Write a program to demonstrate clustering using the K-Means algorithm

Introduction

K-Means is an unsupervised machine learning algorithm used to group similar data points into clusters. K stands for number of clusters. The algorithm groups data based on similarity (distance).

Algorithm

1. Choose number of clusters KKK
2. Initialize KKK centroids randomly
3. Assign each data point to the nearest centroid
4. Recalculate centroids (mean of points in each cluster)
5. Repeat steps 3–4 until centroids stop changing
6. Final clusters are formed

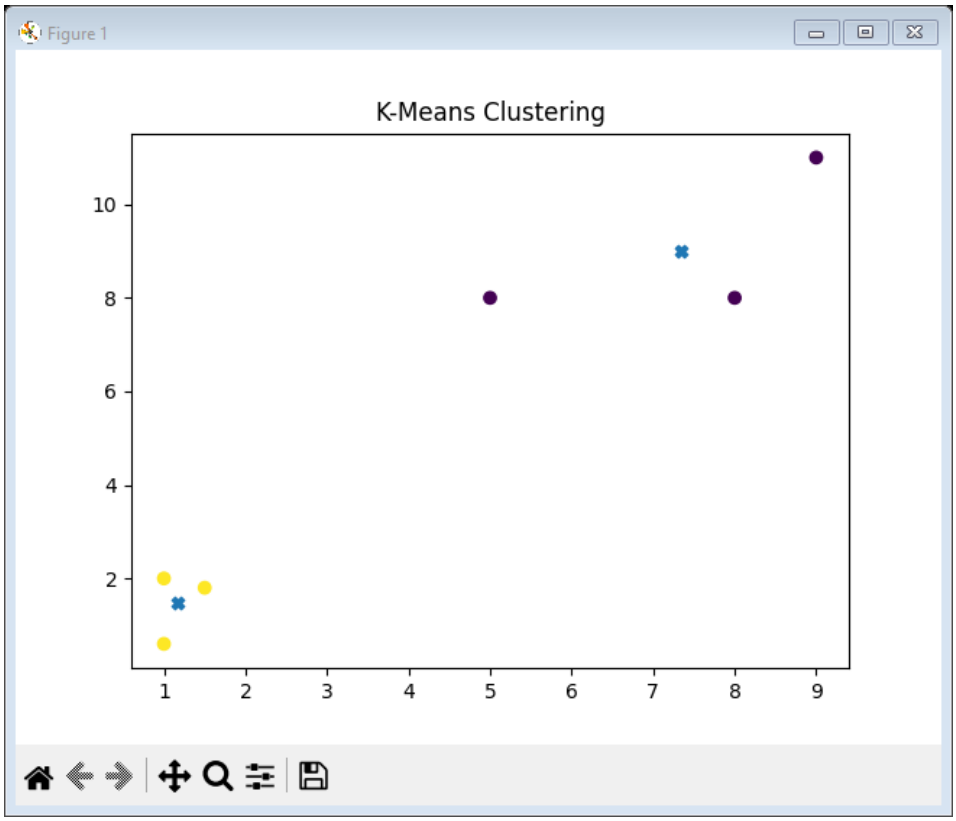
Implementation

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Sample dataset
X = np.array([
    [1, 2], [1.5, 1.8], [5, 8],
    [8, 8], [1, 0.6], [9, 11]
])

# Create K-Means model (K = 2)
kmeans = KMeans(n_clusters=2, random_state=42)
# Fit model
kmeans.fit(X)

# Get cluster labels
labels = kmeans.labels_
# Get centroids
centroids = kmeans.cluster_centers_
# Print results
print("Cluster Labels:", labels)
print("Centroids:\n", centroids)
```

```
# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X')
plt.title("K-Means Clustering")
plt.show()
```

Output

```
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab11-K-MeansAlgorithm.py
Cluster Labels: [1 1 0 0 1 0]
Centroids:
 [[7.33333333 9.        ]
 [1.16666667 1.46666667]]
```



Lab 12: Write a program to demonstrate clustering using the K-Medoids algorithm

Introduction

K-Medoids is an unsupervised machine learning algorithm used for clustering, similar to K-Means. K-Medoids uses *actual data points* as center called medoids**.** K-Medoids is more robust to noise and outliers. K-Medoids has applications in fields like market segmentation, bioinformatics and image grouping.

Algorithm

1. Select number of clusters KKK
2. Randomly choose KKK data points as medoids
3. Assign each point to the nearest medoid
4. For each cluster:
5. Try swapping medoid with another point
6. Calculate total cost (distance)
7. If cost reduces, update medoid
8. Repeat steps 3–5 until no change
9. Final clusters are formed

Implementation

```
import numpy as np
# Sample dataset
X = np.array([
    [1, 2], [2, 3], [3, 4],
    [8, 7], [9, 8], [10, 9]
])

# Step 1: Choose K
k = 2

# Step 2: Initialize medoids randomly
medoid_indices = [0, 3]
medoids = X[medoid_indices]
def calculate_distance(a, b):
    return np.sum(np.abs(a - b))  # Manhattan distance
```

```
# Step 3–6: Iterate
for iteration in range(10):
    clusters = [[] for _ in range(k)]

    for point in X:
        distances = [calculate_distance(point, m) for m in medoids]
        cluster_index = np.argmin(distances)
        clusters[cluster_index].append(point)

    # Update medoids
    new_medoids = []
    for cluster in clusters:
        costs = []
        for candidate in cluster:
            cost = sum(calculate_distance(candidate, p) for p in cluster)
            costs.append(cost)
        new_medoids.append(cluster[np.argmin(costs)])

    new_medoids = np.array(new_medoids)

   # Stop if no change
    if np.array_equal(medoids, new_medoids):
        break
   medoids = new_medoids

print("Final Medoids:\n", medoids)
print("Clusters:")
for i, cluster in enumerate(clusters):
    print(f"Cluster {i+1}: {cluster}")
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab12-K-MedoidsAlgorithm.py
Final Medoids:
 [[2 3]
 [9 8]]
Clusters:
Cluster 1: [array([1, 2]), array([2, 3]), array([3, 4])]
Cluster 2: [array([8, 7]), array([9, 8]), array([10,  9])]
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey>
```

Lab 13: Write a program to demonstrate clustering using Agglomerative (Hierarchical) algorithm

Introduction

Agglomerative Clustering is a hierarchical clustering algorithm (unsupervised learning) that builds clusters in a bottom-up approach. It has application in fields like customer segmentation, document clustering and biology (gene grouping).

Algorithm

1. Treat each data point as a separate cluster
2. Compute distance between all clusters
3. Merge the two closest clusters
4. Update distance matrix
5. Repeat steps 2–4 until desired number of clusters is reached
6. Display final clusters

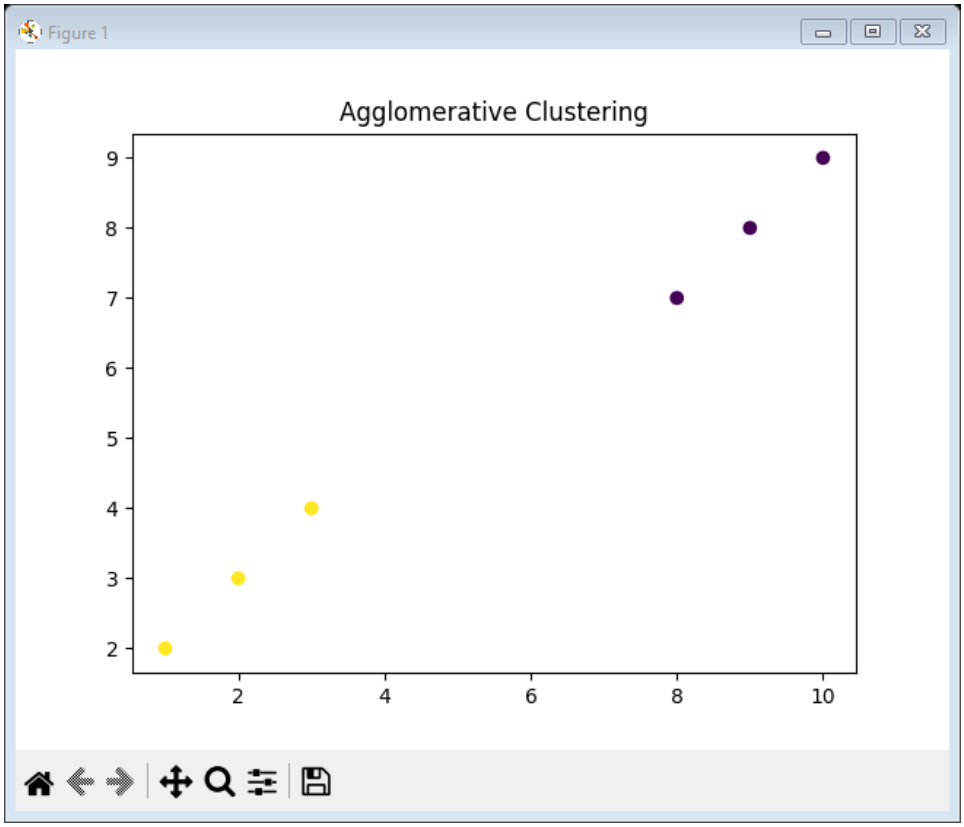
Implementation

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

# Sample dataset
X = np.array([
    [1, 2], [2, 3], [3, 4],
    [8, 7], [9, 8], [10, 9]
])
# Create Agglomerative model
model = AgglomerativeClustering(n_clusters=2, linkage='ward')
# Fit and predict
labels = model.fit_predict(X)
# Print clusters
print("Cluster Labels:", labels)
# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.title("Agglomerative Clustering")
plt.show()
# Create dendrogram
```

```
Z = linkage(X, method='ward')
plt.figure()
dendrogram(Z)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab13-Agglomerative.py
Cluster Labels: [1 1 1 0 0 0]
```



Lab 14: Write a program to demonstrate clustering using DBSCAN algorithm.

Introduction

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an unsupervised clustering algorithm that groups data points based on density**.** DBSCAN does not require number of clusters (K), can find irregular-shaped clusters and can detect noise/outliers DBSCAN can be used in fraud detection, GPS location clustering and image segmentation.

Algorithm

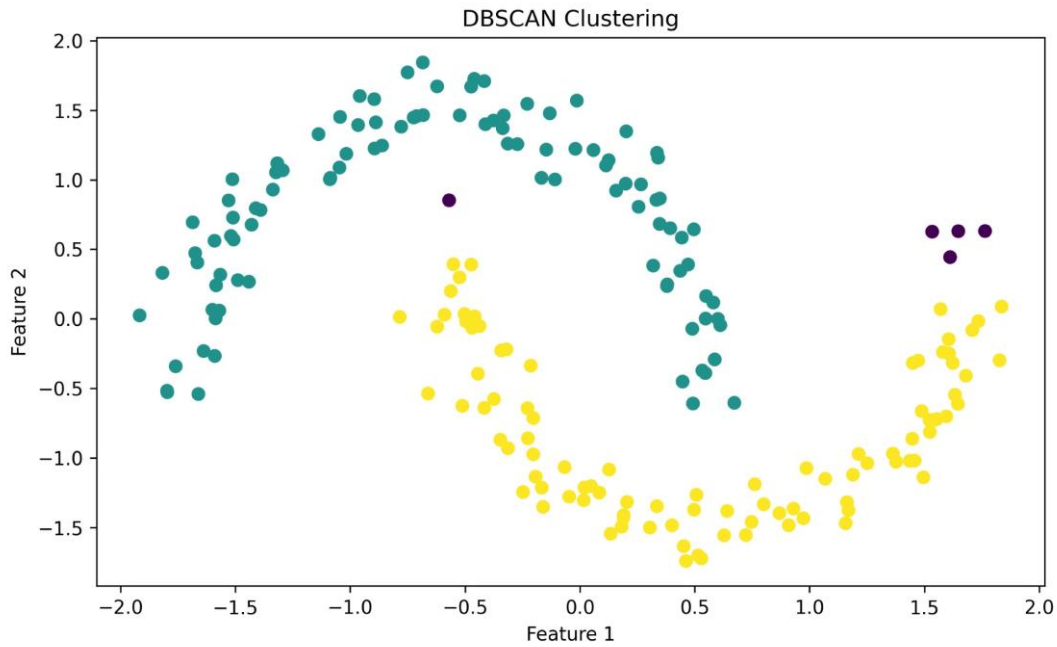
1. Choose two parameters:
    - ε (epsilon) = radius
    - MinPts = minimum points required
2. For each point, find neighbors within ε distance
3. If neighbors ≥ MinPts → mark as **core point**
4. Expand cluster from core points
5. Points not belonging to any cluster → **noise**
6. Repeat until all points are processed

Implementation

```
import numpy as np
class NeuralNetwork:
    def __init__(self, input_size: int, hidden_size: int, output_size: int,
                 learning_rate: float = 0.5):
        rng = np.random.default_rng(42)
        self.learning_rate = learning_rate
        self.w1 = rng.normal(scale=1.0, size=(input_size, hidden_size))
        self.b1 = np.zeros((1, hidden_size))
        self.w2 = rng.normal(scale=1.0, size=(hidden_size, output_size))
        self.b2 = np.zeros((1, output_size))

    @staticmethod
    def sigmoid(x):
        return 1 / (1 + np.exp(-x))
```

```
    @staticmethod
    def sigmoid_derivative(output):
        return output * (1 - output)

    def forward(self, x):
        self.z1 = np.dot(x, self.w1) + self.b1          # FIX: [np.dot](http://np.dot) → np.dot
        self.a1 = self.sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.w2) + self.b2    # FIX: [np.dot](http://np.dot) → np.dot
        self.a2 = self.sigmoid(self.z2)
        return self.a2

    def train(self, x, y, epochs: int = 10000):
        for epoch in range(epochs):
            output = self.forward(x)
            error = y – output
            loss = np.mean(np.square(error))
            d_output = error * self.sigmoid_derivative(output)
            d_hidden = np.dot(d_output, self.w2.T) * self.sigmoid_derivative(self.a1)  # FIX:
[np.dot](http://np.dot) → np.dot

            self.w2 += np.dot(self.a1.T, d_output) * self.learning_rate              # FIX:
[np.dot](http://np.dot) → np.dot
            self.b2 += np.sum(d_output, axis=0, keepdims=True) * self.learning_rate
            self.w1 += np.dot(x.T, d_hidden) * self.learning_rate                    # FIX:
[np.dot](http://np.dot) → np.dot
            self.b1 += np.sum(d_hidden, axis=0, keepdims=True) * self.learning_rate

            if epoch % 2000 == 0:
                print(f"Epoch {epoch:5d} | Loss: {loss:.6f}")
    def predict(self, x):
        return self.forward(x)
def main() -> None:
    x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
    y = np.array([[0], [1], [1], [0]], dtype=float)

    model = NeuralNetwork(input_size=2, hidden_size=2, output_size=1,
                          learning_rate=0.5)
    model.train(x, y, epochs=10000)
    predictions = model.predict(x)
    print()
    print("Final predictions:")
    for sample, prediction in zip(x, predictions):
        print(f"Input: {sample.astype(int).tolist()} -> Output: {prediction[0]:.4f}")
```

```
if __name__ == "__main__":
    main()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL
TERMINAL
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey> python lab14-DBSCAN.py
Input: [1, 0] -> Output: 0.9752
Input: [1, 1] -> Output: 0.0201
PS C:\Users\User\OneDrive\Desktop\dwdm- SamchitaPandey>
```